

COMS4705 Final Exam Review

Noah, Keyang, Sayali

Agenda

1. **Machine Learning for NLP (Noah)**
2. **Deep learning for NLP, Pre-trained LMs and Applications (Keyang)**
3. **Lexical Semantics (Sayali)**
4. **Machine translation (Sayali)**

Machine Learning for NLP

Noah Bergam njb2154

Overview

Mechanics of Vanilla Neural Network

Backprop, loss, and SGD

Neural language models

True/False

1. Neural Networks can act as both generative and discriminative classifiers.
2. Too much regularization leads to overfitting.
 - a. Recall dropout regularization
3. The perceptron algorithm will always converge if the data is linear separable.
4. HMM is discriminative. MEMM is generative.
5. Neural language models learn embeddings as they model next-word probabilities.
6. Gradient descent is called stochastic (SGD) if we make gradient updates based on the entire dataset, rather than subsets or single data points.
7. One-hot embeddings of words reflect the distributional hypothesis.

Word Embeddings

Static vs. dynamic embeddings; Geometry of Analogies

One-hot encoding

- Major issues?

SVD and co-occurrence matrix

TF-IDF

- How to compute?

Word2Vec

- Differences between skip-gram and CBOW
- Why does skip-gram need negative sampling.

Skip-Gram vs CBOW

- Two different objectives for learning embeddings:
 - **Skip-Gram model:** Input is a single word.
Goal: Predict a probability for each context word.
 - **Continuous bag-of-words (BOW):**
Input is a representation of the context window.
Goal: Predict a probability for each target word.
- Use either **Negative Sampling** or Hierarchical Softmax for training.
- Inspired by Neural Language Models (**Bengio et al. 2003**)

Skip-gram with Negative Sampling

Softmax is computationally difficult. Why?

softmax activation

$$P(D|w, c) = \frac{1}{1 + e^{-score(w, c)}}$$

$$P(\bar{D}|w, c) = 1 - P(D|w, c) = 1 - \frac{1}{1 + e^{-score(w, c)}}$$

- Objective is to maximize

$$\sum_{(w, c) \in D} \log P(D|w, c) + \sum_{(w, c) \in \bar{D}} \log P(\bar{D}|w, c)$$

$$\begin{aligned} P(w|eat) \\ &= \frac{\exp(score(eat, w))}{\sum_{v \in V} \exp(score(eat, v))} \end{aligned}$$

Algorithm Practice

3. (15 points) Below is the Viterbi algorithm. Assume that the tags are indexed into the set of integers from 1 to t .

Algorithm 1 Viterbi Algorithm

```
1: function VITERBI( $x, S, T, E$ )   $\triangleright x$ : sentence of length  $n$ ,  $S$ : initial log probs,  $T$ : transition log probs,
    $E$ : emission log probs
2:   Initialize  $v$ , a  $n \times t$  matrix
3:   for  $y = 1$  to  $t$  do  $\triangleright$  Handle the initial state
4:      $v[1, y] = S[y] + E[y, x_1]$ 
5:   end for
6:   for  $i = 2$  to  $n$  do
7:     for  $y = 1$  to  $t$  do
8:        $v[i, y] = E[y, x_i] + \max_{y_{\text{prev}}} (T[y, y_{\text{prev}}] + v[i - 1, y_{\text{prev}}])$ 
9:     end for
10:  end for
11:  Best final state =  $\arg \max_y (v[n, y] + T[\text{STOP}, y])$ 
12:  Reconstruct answer with tracked argmaxes (not shown)
13: end function
```

a. (2 points) What is the runtime of the Viterbi algorithm?

b. (4 points) Suppose certain state transitions are completely illegal; that is, $P(y|y_{\text{prev}}) = 0$ for those state pairs. Modify the above algorithm to take advantage of this information. You should write your answer as a description of which line numbers to modify and either a rewriting of the appropriate lines or a clear, precise textual description of what needs to change.

Two possibly useful notions we will define for you: $N(y)$ = the set of tags that can legally precede y . $B(y)$ = set of tags that can legally follow y .

Deep Learning for NLP, Pre-trained LM and Applications

Keyang Xu
keyang.xu@columbia.edu

You need to ...

- Understand the **high-level ideas** of different neural networks, e.g., LSTM and transformers
- Know what are pre-trained LMs, and the **different types of pre-training methods** (e.g., ELMo, BERT and GPT)
- The **pipelines** of using different pre-trained LMs.
- Have the ability to design NLP systems for different applications.

Questions in the final exam will

- **Be** short answer questions, and might be open-ended (for design questions)
- **Not** focus on implementations based on specific libraries
- **Not** focus on memorizing details, e.g., the equation of each gate in LSTM, every component in a transformer block, or the hyperparameters for BERT...

RNNs and LSTMs

- **Key idea**: Modeling the input text **sequentially**. The hidden representation of the previous state is taken as input for modeling the next state.
- Problem in training RNN: **Gradient vanishing**
- Attention mechanism: simply weighted sum of representations
- Acceptor v.s. Encoder v.s. Transducer. What are their suitable applications?

RNNs and LSTMs - Sample Questions

1. Why do we prefer LSTM over RNN? Which one is better at classifying long documents?
2. Describe the attention mechanism in a LSTM-based Seq2Seq model
3. Describe how to stack multiple layers of LSTMs.

Transformers

- **Key idea:** No sequential modeling. Use **self-attention** to get **contextualized embedding**.
- Again, self-attention is still a weighted sum of features representations. The major components are Key, Query, Value matrices.
- Three types of self-attention: (1) encoder attention; (2) decoder attention; (3) encoder-decoder attention
- We can stack multiple transformer encoders and decoders.

Transformers - Sample Questions

1. What are the differences between a transformer encoder and decoder?
2. Comparing LSTMs and Transformers: (i) which model is better at parallelization, and why? (ii) which model is better at modeling long-range dependencies? (iii) which model has better time complexity regarding document length?
3. Does transformer use any position information?
4. What is the multi-head attention in a transformer?

Pre-trained Language Models

- **Key idea**: using self-supervised objectives to train a LM on large-scale general corpora
- **Different training objectives:**
 - **Masked language model (MLM) and Next sentence prediction (NSP) in BERT-family**
 - MLM: Randomly mask tokens from a text input and reconstruct them using a **transformer encoder**
 - NSP: Binary classification for whether two sentences are consecutive in context
 - **Auto-regressive text generation in GPT-family**
 - Predicting each word given the previous words on the left using a **transformer decoder**

Pre-trained Language Models - Sample Questions

1. What do ELMo and Word2vec share in common? What are their differences?
2. Differences between BERT and GPT
3. In MLM, can you describe the output vector after softmax layer? How to optimize the LM using this output?
4. In NSP, how do you create positive and negative training samples?

Pipeline of using pre-trained LM for NLP applications

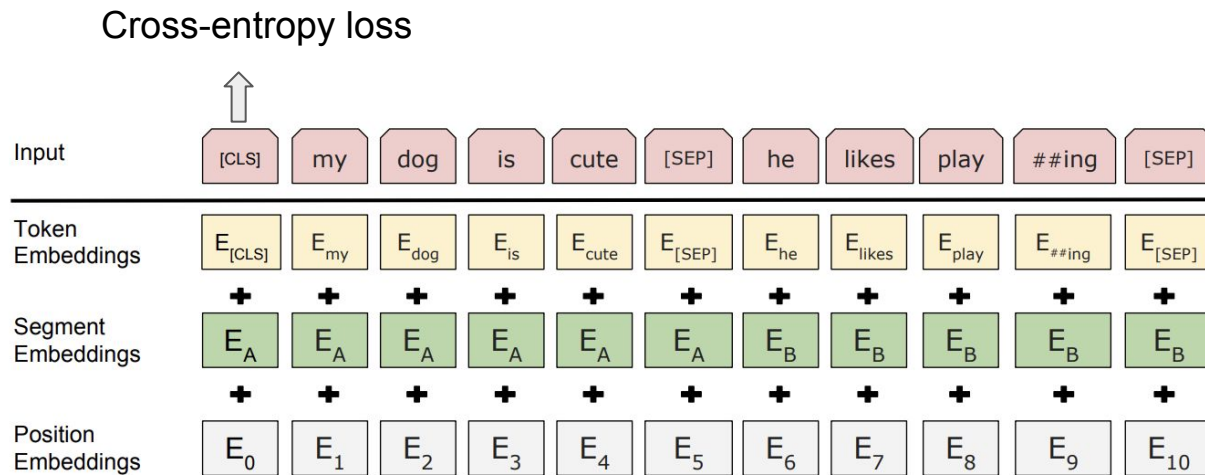
- ELMo: pre-trained RNN + extracting contextualized embedding
- BERT: pre-trained bidirectional transformer + fine-tuning
- GPT-3: pre-trained unidirectional transformer + few-shot in-context learning

NLP system design - Open questions

- How to use a BERT model to classify if two given sentences into *entailment*, *contradiction*, or *neutral*, given enough supervised training data?
- ❖ Key component 1: how to combine two sentences as one sequence?
- ❖ Key component 2: how to represent the sequence in text classification?
- ❖ Key component 3: What is the loss function in optimization?

NLP system design - Open questions

- How to use a BERT model to classify if two given sentences are *entailment*, *contradiction*, or *undetermined*, given enough supervised training data?



NLP system design - Open questions

- How to use a BERT model to classify if two given sentences are *entailment*, *contradiction*, or *undetermined*, given enough supervised training data?
- **How about doing a supervised NER task?**
 - ❖ Make sure you understand the Figure 1 & 2 in the BERT paper.

Lexical Semantics

Vocabulary: Define each

Homonymy: bank of river vs bank of America

Polysemy: The history of man vs that tall man

Metonymy: suit = business executive

Zeugma: Does United serve breakfast and JFK?

Synonymy / Antonymy: hot / cold

Hyponymy / Hypernym: dog / mammal, desk / furniture

Meronymy / holonym: wheel / car

Machine Translation: Statistical

Understanding Alignments and IBM Model 2

e: And the programme has been implemented

f: Le programme a ete mis en application

Alignment variables: specify an alignment for each French word to some word in the English sentence

$$\langle a_1, a_2, a_3, \dots \rangle \quad a_1, a_2, \dots, a_7 = \langle 2, 3, 4, 5, 6, 6, 6 \rangle$$

We have to model $p(f|e)$

How many possible alignments in A?

- How many possible alignments for (f, e) where
 - f is French sentence with m words
 - e is an English sentence with l words
- For each of m French words, we choose an alignment link among $(l+1)$ English words
- Answer: $(l + 1)^m$

Parameters

- $t(f|e)$
 - Word translation probability table
 - for all words in French & English vocab

f	e	$p(f e)$
le	the	0.42
la	the	0.4
programme	the	0.001
a	has	0.78
...	...	...

Parameters

- $q(j|i,l,m)$
 - now a table
 - not uniform as in IBM1
- How many parameters are there?

j	$q(j 1, 6, 7)$
1	0.27
2	0.14
...	...
48	1E-75

Inference

Inference

- Given
 - a sentence pair (e, f)
 - an alignment model with parameters $t(f|e)$ and $q(j|i, l, m)$
- What is the most probable alignment a ?

Use these parameters to compute:

$$P(F|E) = \sum_A P(F, A|E)$$

Hint: recall independence assumptions!

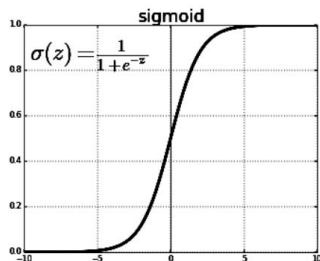
**HOMEWORK: PRACTICE
UNGRADED EXERCISE (IBM
Model 2)**

$$P(F|E) = \prod_{i=1}^m q(a_i|i) t(f_i|e_{a_i})$$

Extra Problems

Extra Practice Problems: Activation functions and motivation of LSTM

(c) Consider the sigmoid activation function below:

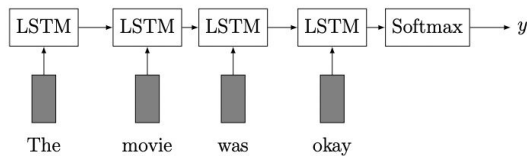


- i. (2 points) What would the gradient of the sigmoid be with respect to an input that is very large?

- ii. (2 points) Why might this be a problem for training neural networks?

- iii. (2 points) How does the ReLU activation $\text{ReLU}(z) = \max(0, z)$ solve this problem?

(d) A popular model used for sentiment classification is an LSTM model:



This model inputs word vectors to the LSTM model at each time step and uses the last hidden state vector to predict the sentiment label (y).

Recall that in assignment 1 we used a simple “bag-of-vectors” model for sentiment classification: we used the average of all the word vectors in a sentence to predict the sentiment label.

- i. (3 points) Name at least one benefit of the LSTM model over the bag-of-vectors model.

- ii. (3 points) If we chose to update our word vectors when training the LSTM model on sentiment classification data, how would these word vectors differ from ones not updated during training? Explain with an example. Assume that the word vectors of the LSTM model were initialized using GloVe or word2vec.

Answers

ci) zero / close to zero - slope gets small

cii) this would lend itself to the vanishing gradient problem — this might lead the model to not learn

ciii) for very large inputs, the gradient is unchanged (because derivative of this is zero); however, still nonlinear

di) averaging the word vectors loses sequential structure; LSTM hidden state is built sequentially, will pay more attention to recent words

dii) the word embeddings will be affected (fine-tuned, in some sense) by the sentiment classification task. For instance, words with similar sentiments like “good” and “happy” will be closer together; “bad” and “smile” might be further.

RNN

5. Recurrent neural networks (20 points)

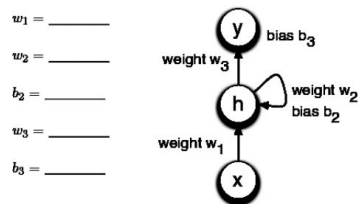
Recall that a recurrent neural network (RNN) takes in an input vector x_t and a state vector h_{t-1} and returns a new state vector h_t and an output vector y_t :

$$h_t = f(w_1 x_t + w_2 h_{t-1} + b_2)$$

$$y_t = g(w_3 h_t + b_3),$$

where f and g are activations functions.

- (a) (8 points) The following diagram depicts a single RNN unit, where x_t , h_{t-1} , h_t and y_t are all scalars, as a state machine:



Suppose that f is a binary threshold unit and g is a linear unit:

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

$$g(x) = x.$$

Fill in weights (w_1, w_2, w_3) , biases (b_1, b_2) so that the RNN initially outputs 0, but as soon as it receives an input of 1, it switches to outputting 1 for all subsequent time steps. For instance, the input 0001010 produces the output 0001111. The hidden unit has an initial value of 0.

You don't need to provide an explanation, but doing so may help you receive partial credit.

Hint: In one possible solution, the hidden unit has an activation $h_t = 0$ until there's an input $x_t = 1$, at which point it switches to maintaining an activation of 1 forever. The output unit always predicts the same as the hidden unit, i.e. $y_t = h_t$.

ANSWER

$$W_3 = 1$$

$$W_2 = 1$$

$$W_1 = 1$$

$$B_2 = -1$$

$$B_3 = 0$$

Hidden state starts at zero; until we introduced to a 1, $y = g(w_3h + b_3) = g(0)=0$.

Once we see a 1, hidden state becomes 1 and it stays at 1.

5. Recurrent neural networks (20 points)

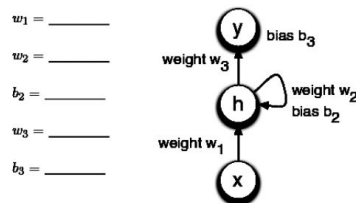
Recall that a recurrent neural network (RNN) takes in an input vector x_t and a state vector h_{t-1} and returns a new state vector h_t and an output vector y_t :

$$h_t = f(w_1x_t + w_2h_{t-1} + b_2)$$

$$y_t = g(w_3h_t + b_3),$$

where f and g are activations functions.

- (a) (8 points) The following diagram depicts a single RNN unit, where x_t , h_{t-1} , h_t and y_t are all scalars, as a state machine:



Suppose that f is a binary threshold unit and g is a linear unit:

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$
$$g(x) = x.$$

Fill in weights (w_1, w_2, w_3), biases (b_1, b_2) so that the RNN initially outputs 0, but as soon as it receives an input of 1, it switches to outputting 1 for all subsequent time steps. For instance, the input 0001010 produces the output 0001111. The hidden unit has an initial value of 0.

You don't need to provide an explanation, but doing so may help you receive partial credit.

Hint: In one possible solution, the hidden unit has an activation $h_t = 0$ until there's an input $x_t = 1$, at which point it switches to maintaining an activation of 1 forever. The output unit always predicts the same as the hidden unit, i.e. $y_t = h_t$.

Dep Parsing

2. Short questions (24 points)

Please write answers to the following questions in a sentence or two. Each part is worth 6 points.

(a) Dependency Parsing

- i. (2 points) Can the neural dependency parser we learned in class correctly parse the sentence “John saw a dog yesterday which was a Yorkshire terrier.”? Explain.

- ii. (2 points) What is the ambiguity in parsing the sentence “There’s an awful cost to getting a PhD that no one talks about”?

- iii. (2 points) Name at least two types of features that would be helpful to provide as input to a neural dependency parser and explain why.